실습 준비 (공통 checkpoint 확인)

> 검증된 공통 checkpoint에서 시작해 전날 오류가 오늘 통합으로 전파되지 않게 합니다.

6일차 통합 실습은 개인 결과물이 아니라 미리 검증된 공통 checkpoint에서 시작합니다. 교육 패들릿에 올려 둔 `day6_checkpoint.zip` 을 내려받아 `day6_practice` 폴더에 풀고 시작합니다.
안에는 2일차에 만들어 디스크에 저장해 둔 벡터 DB와 4일차 MCP 서버가 들어 있습니다.

임직원, 자산, 프로젝트 데이터는 checkpoint에 넣지 않았습니다. 4일차에 만든 세 JSON을 그대로 가져옵니다. 없으면 4일차 `make_data.py` 를 다시 실행하면 되고 결과는 항상 같습니다.

단 `eval_set.json` 은 예외입니다. 본인 GitHub 레포(sds-ax-practice)에 있는 파일이 정본입니다. 레포에서 실습 폴더로 복사해 쓰고 끝나면 다시 push합니다.
checkpoint에 들어 있는 `eval_set.json` 은 전날 도전 미션을 따라오지 못한 경우에만 씁니다. 그래야 3일차부터 5일차까지 본인이 추가한 문항을 잃지 않습니다.

따라하기

1. 새 실습 폴더를 만들고 패들릿에서 받은 `day6_checkpoint.zip` 을 풀어 필수 패키지를 설치합니다.
   1~5일차는 레포 루트의 가상환경을 계속 썼지만, 오늘은 checkpoint 코드에 맞춰 **이 폴더에 새 가상환경을 따로 만듭니다**(앞서 만든 환경은 그대로 두세요). 7일차는 오늘 만드는 이 환경을 그대로 이어 씁니다.

```
mkdir day6_practice
cd day6_practice
# 패들릿에서 받은 day6_checkpoint.zip을 이 폴더에 두고 풉니다
Expand-Archive day6_checkpoint.zip -DestinationPath .
# 탐색기에서 마우스 오른쪽 버튼으로 "압축 풀기"를 해도 됩니다
python -m venv .venv
.venv\Scripts\activate
python -m pip install -r ..\requirements.txt
```

2. 4일차에 만든 JSON 3종을 이 폴더(day6_practice)로 복사한 뒤 구조를 확인합니다. `.env`는 레포 루트의 것을 그대로 쓰므로 복사하지 않습니다.

```
# day6_practice/ 구조
# ./chroma_db/          2일차 벡터 DB (사내 정책 문서 임베딩 완료)
# ./mcp_server.py       4일차 MCP 서버
# ./employees.json      4일차에서 make_data.py로 만든 임직원 데이터 (checkpoint에 없음)
# ./assets.json         4일차에서 make_data.py로 만든 자산 데이터 (checkpoint에 없음)
# ./projects.json       4일차에서 make_data.py로 만든 프로젝트 데이터 (checkpoint에 없음)
#                       세 파일은 4일차 폴더에서 복사해 오고 없으면 make_data.py를 다시 실행하면 결과는 같습니다
# ./eval_set.json       2일차부터 누적한 평가셋
#                       Bedrock 자격 증명(.env)은 레포 루트에 있습니다
```

3. 벡터 DB와 MCP 서버가 살아 있는지 조립 전에 단독으로 먼저 확인합니다.

```
# check_checkpoint.py - checkpoint 상태 점검
from pathlib import Path

from dotenv import load_dotenv
from langchain_aws import BedrockEmbeddings
from langchain_chroma import Chroma

load_dotenv()

# 이 세 파일은 checkpoint에 없습니다. 4일차에서 make_data.py로 만든 것을 가져왔는지 먼저 확인합니다
for name in ["employees.json", "assets.json", "projects.json"]:
    status = "있음" if Path(name).exists() else "없음 -> 4일차의 make_data.py로 다시 만드세요"
    print(f"{name}: {status}")

# 2일차와 동일한 임베딩 모델이어야 합니다 (다르면 오류 없이 이상한 결과가 나옵니다)
embeddings = BedrockEmbeddings(
    model_id="amazon.titan-embed-text-v2:0",
    region_name="us-east-1",
)
vectorstore = Chroma(
    persist_directory="./chroma_db",
    embedding_function=embeddings,
    collection_name="sds_policies",
)

print("벡터 DB 문서 수:", vectorstore._collection.count())
docs = vectorstore.similarity_search("출장비 정산", k=2)
for d in docs:
    print("-", d.metadata.get("source"), "|", d.page_content[:40])
```

```
# MCP 서버도 단독 기동이 되는지 확인 (Ctrl+C로 종료)
python mcp_server.py
```

막힐 때

- 문서 수가 0으로 나옵니다: `persist_directory` 경로와 `collection_name`이 checkpoint와 같은지 확인합니다. 새 폴더를 쓰면 Chroma가 빈 DB를 새로 만들어 오류 없이 0이 출력됩니다.
- 임베딩 모델을 다른 것으로 바꿨다면 검색 결과가 엉뚱해집니다: 이 경우 오류 메시지가 나지 않으므로 가장 찾기 어려운 문제입니다. 2일차와 동일한 `amazon.titan-embed-text-v2:0`를 그대로 쓴다고 생각하지 말고 코드로 한 번 확인합니다.

- `python mcp_server.py`가 바로 종료됩니다: JSON 데이터 파일이 같은 폴더에 없을 때의 `FileNotFoundError`입니다. 세 파일은 checkpoint에 들어 있지 않으므로 4일차 폴더에서 복사해 오거나 4일차 `make_data.py`를 다시 실행해 만듭니다.

확인: 문서 수가 0이 아니고 검색 결과에 사내 정책 문서 문장이 보이며 MCP 서버가 오류 없이 기동하면 성공입니다.

> 오늘 조립할 재료가 모두 살아 있는지 먼저 확인하는 순서를 익혔습니다.
